

Electric Taxi Management System

System Flow & Technical Design Document

Prepared for: Sindh Government — Electric Taxi Initiative

Prepared by: iTechnologi Group

Document Version: 1.0 (Draft for Review)

Date: July 2026

1. Project Overview

The Sindh Government's Electric Taxi initiative requires a ride-hailing platform, similar in concept to Uber, built specifically around electric taxis operating within defined geo-fenced zones. The system connects passengers, drivers, and administrators through three coordinated applications, backed by a real-time GPS tracking layer installed on every vehicle.

This document translates the requirements shared into a structured system flow covering vehicle tracking, geo-fencing and validation, the ride lifecycle, fare calculation, and payments — intended as a working reference for both the client and the development team.

1.1 Core Requirements Summary

- GPS trackers installed on all electric taxi vehicles for continuous location monitoring.
- Admin-defined Geo-Fencing zones that vehicles are assigned/paired to.
- Passenger mobile app to request a ride from within a geo-fenced area, without a visible upfront fare.
- Driver (Android) panel to accept requests, start the ride, and end the ride.
- Fare calculated automatically using time and distance travelled (formula to be provided by client).
- Multiple payment methods: Cash, JazzCash, and Card.
- Vehicle-to-geofence pairing validation, with automated warnings if a vehicle is detected in a different geo-fence area without an assigned activity/trip.

2. System Architecture — Components

The platform is composed of five coordinated components. Each is described below along with its primary responsibility.

Component	Responsibility
Passenger App (Android/iOS)	Ride booking, live driver tracking, ride status, in-app payment, ride history, ratings.
Driver App (Android Panel)	Accept/reject ride requests, navigation, start/end ride, view earnings, geofence status.
Admin Web Panel	Geofence creation/management, vehicle-geofence pairing, driver/vehicle onboarding, fare formula config, monitoring, violation alerts, reporting.
Vehicle Tracking Layer (IoT/GPS)	Hardware trackers on each vehicle sending continuous GPS pings to the backend via GSM/cellular network.
Core Backend & Dispatch Engine	Ride matching, geofence validation, fare engine, payments, notifications (push/SMS), central database.

3. Ride Lifecycle — Step-by-Step Flow

This is the primary passenger-to-driver flow, from ride request through to payment and closure.

Step 1 — Passenger Requests Ride

Passenger opens the app inside a geo-fenced service area. Pickup location is auto-detected (GPS) or manually pinned. No fare estimate is shown at this stage, per requirement. Request is sent to the dispatch engine.

Step 2 — Dispatch & Driver Matching

Backend calculates the 3 geofence zones nearest to the pickup point by straight-line distance (the passenger's own zone may or may not be among them, depending on geography). A ride notification is broadcast simultaneously to all available (online, free) vehicles within these 3 zones. The first driver to tap 'Accept' is locked to the ride; the request is automatically withdrawn from all other drivers. See Section 3.1 for the fallback behaviour when no vehicle is available.

Step 3 — Driver En Route to Pickup

Passenger sees live driver location and ETA. Driver navigates to pickup point using in-app maps. Ride status: 'Driver Assigned' → 'Arriving'.

Step 4 — Ride Start (Trip Begins)

On reaching the passenger, the driver taps 'Start Ride' on the Android panel. This timestamp and GPS point mark the official trip-start (t_0 , distance = 0). Status: 'In Progress'.

Step 5 — Live Trip Tracking

Backend continuously logs GPS coordinates and timestamps from the vehicle tracker to compute running distance and elapsed time; passenger sees live route on the app.

Step 6 — Ride End (Trip Closes)

On reaching the destination, the driver taps 'End Ride'. Trip-end timestamp and location are recorded; total time and distance are finalized.

Step 7 — Fare Calculation

The fare engine applies the client-provided formula against final time and distance to compute the total fare. (Formula to be supplied — see Section 5.)

Step 8 — Payment

Passenger settles the fare via Cash, JazzCash, or Card. On confirmation, the ride is marked 'Completed' and a digital receipt is generated.

Step 9 — Post-Ride

Both passenger and driver may rate the trip. Ride record, GPS trail, fare breakdown, and payment status are archived for reporting and dispute handling.

3.1 Zone Expansion Logic (No Driver Available)

If none of the initial 3 nearest zones have an available vehicle, the dispatch engine automatically widens the search rather than failing immediately:

- Round 1: Notify all available vehicles in the 3 nearest zones (by distance from pickup point).
- Round 2: If no acceptance within the matching window, expand to the 4th nearest zone and re-broadcast.
- Round 3+: Continue expanding one zone at a time (5th, 6th...) until an available vehicle is found or a maximum search radius / zone count (configurable by admin) is reached.
- Fallback: If expansion reaches its configured limit with still no available vehicle, the passenger is shown a 'No driver available right now' message and may be prompted to retry.

4. Geo-Fencing & Vehicle Validation

Geo-fencing is central to how vehicles are permitted to operate and how anomalies are detected.

4.1 Geofence Setup

- Admin draws/defines a geo-fence boundary (polygon or radius) on the Admin Panel map for each operating zone.
- Each vehicle is assigned (paired) to one primary geofence zone at onboarding.
- A vehicle may only accept ride requests that originate and/or operate within its paired geofence, unless explicitly reassigned by the admin.

4.2 Violation Detection Flow

Stage	Description
Continuous Monitoring	The tracking layer reports vehicle GPS position at regular intervals to the backend.
Zone Check	Each position ping is checked against the vehicle's paired geofence boundary.
Outside-Zone Detected	If a vehicle is found outside its paired geofence, the system checks whether it is currently on an active/assigned trip.
No Active Activity	If there is no active ride, dispatch record, or admin-approved reassignment tied to that movement, the vehicle is flagged.
Warning Issued	An automated warning/alert is generated — visible to the Admin Panel and optionally pushed to the driver — logging vehicle ID, timestamp, location, and duration outside zone.
Escalation (configurable)	Repeated or prolonged violations can trigger escalation rules, e.g. supervisor notification or temporary suspension, as defined by the client.

5. Fare Calculation Module

Fare will be computed as a function of ride duration and distance travelled, both measured from the GPS trail captured between 'Start Ride' and 'End Ride'.

- Inputs: total distance (km, from GPS trail), total time (minutes, from start/end timestamps), and the client-provided fare formula (e.g. base fare + per-km rate + per-minute rate, with possible surge/zone multipliers).
- The formula itself is a pending input from the client — the fare engine will be built as a configurable module so rates can be updated from the Admin Panel without code changes.
- Fare is calculated only after 'End Ride' is triggered, so nothing is shown to the passenger before booking, per the requirement.
- A fare breakdown (base + distance + time + any adjustments) should still be shown on the receipt after the ride, for transparency and dispute resolution.

6. Payment Module

Method	Flow
Cash	Driver manually confirms cash received on the Android panel; system marks ride as paid.
JazzCash	Passenger pays via JazzCash API/checkout at ride-end; backend confirms transaction before closing the ride as paid.
Card	Passenger pays via integrated card payment gateway; standard authorization/capture flow at ride-end.

7. Core Data Entities (High-Level)

A starting point for database design — to be refined during technical design phase.

Entity	Key Fields
Passenger	id, name, phone, email, saved payment methods, ride history
Driver	id, name, phone, license info, assigned vehicle, status (online/offline/busy)
Vehicle	id, plate no., tracker device ID, paired geofence ID, status
GeofenceZone	id, name, boundary polygon/radius, active status
Ride	id, passenger ID, driver ID, vehicle ID, pickup/drop coordinates, start/end timestamps, distance, duration, fare, payment method, status
GPSLog	id, vehicle ID, latitude, longitude, timestamp, speed
GeofenceViolation	id, vehicle ID, geofence ID, detected time, duration, resolved status
Payment	id, ride ID, method, amount, transaction reference, status

8. Suggested Technology Stack

Layer	Recommendation
Passenger & Driver Apps	Flutter (single codebase for Android/iOS) or native Android for the driver panel
Admin Web Panel	React / Next.js
Backend API	NestJS (Node.js) — REST/WebSocket for real-time tracking
Database	PostgreSQL with PostGIS extension (for geofence/geo-queries)
Real-Time Location	WebSockets / MQTT for live GPS streaming from trackers
Payments	JazzCash API integration + card gateway (e.g. HBL/1LINK-compliant gateway)
Infrastructure	Docker containers, PM2 process management, Linux VM/cloud hosting
Notifications	Firebase Cloud Messaging (push) + SMS gateway

9. Non-Functional Requirements

- Real-time or near-real-time GPS updates (target: every 5–10 seconds per vehicle).
- High availability of the dispatch engine — ride matching should complete within a few seconds.
- Data security for passenger PII and payment transactions (encryption in transit and at rest).
- Audit trail for every ride, geofence violation, and payment transaction.
- Scalable architecture to support growth in vehicle fleet and service zones across Sindh.

10. Pending Inputs / Open Items

- Exact fare calculation formula (base fare, per-km rate, per-minute rate, surge rules, if any).
- List and boundaries of initial geo-fence zones to be launched.
- JazzCash merchant account / API credentials and card payment gateway selection.
- Vehicle tracker hardware specification and communication protocol (for backend integration).
- Violation escalation policy (warning-only vs. suspension thresholds).

This document is a working draft intended to align on system flow before detailed technical design and development begin.